

Quiz 3

● Graded

Student

Luis Méndez Lázaro

Total Points

9.5 / 26 pts

Question 1

Learning rate schedule

1 / 1 pt

✓ + 1 pt Correcto!

+ 0 pts Respuesta: b) *Acelera la convergencia inicial y ajusta finamente el LR cerca de óptimos.*

El warm-up permite que el learning rate (LR) aumente progresivamente, facilitando una convergencia más rápida al inicio del entrenamiento (ayuda a descubrir la dirección adecuada para aproximarnos al mínimo global). Posteriormente, el cosine decay reduce suavemente el LR, permitiendo ajustes finos cerca de los óptimos, lo que no se logra con el enfoque abrupto de step decay.

Question 2

ROI align vs ROI pooling

1 / 1 pt

✓ + 1 pt Correcto!

+ 0 pts Respuesta: a) *Elimina la cuantización de coordenadas mediante interpolación bilineal.*

ROI Align utiliza interpolación bilineal para mapear con mayor precisión las coordenadas de las regiones de interés a los feature maps, evitando la cuantización de coordenadas que introduce ROI Pooling. Esto mejora la alineación espacial y la precisión de las predicciones.

Question 3

Representacion equivariante

0 / 1 pt

+ 1 pt Correcto!

✓ + 0 pts Respuesta: c) *La representación refleja la misma transformación aplicada a la entrada, permitiendo mantener información sobre la variación estructural.*

Una representación equivariante asegura que al aplicar una transformación a la entrada, la representación resultante se transforme de manera correspondiente. Esto permite que la red mantenga información sobre cómo varía estructuralmente la entrada bajo diferentes transformaciones.

Question 4

Transposed convolution

0 / 1 pt

+ 1 pt Correcto!

✓ + 0 pts Respuesta: d) *Introduce artefactos (checkerboard artifacts).*

Las transposed convolutions pueden generar patrones no deseados conocidos como checkerboard artifacts debido a la forma en que realizan el upsampling, lo que puede afectar la calidad de las reconstrucciones.

Question 5

DETR

1 / 1 pt

✓ + 1 pt Correcto!

+ 0 pts **Respuesta: e)** *Servir como un conjunto fijo de embeddings que es procesado con la salida del encoder para predecir las bounding boxes y las clases.*

Los object queries son un conjunto fijo de embeddings que interactúan con la salida del encoder a través del decoder para predecir las bounding boxes y las clases de los objetos presentes en la imagen.

Question 6

Equivarianza a las traslaciones

0 / 1 pt

+ 1 pt Correcto!

✓ + 0 pts **Respuesta: e)** *CNN sin max-pooling.*

Las Convolutional Neural Networks (CNNs) son inherentemente equivariantes a traslaciones debido a la operación de convolución, que aplica los mismos filtros a lo largo de toda la entrada.

Question 7

Generalization

0 / 1 pt

+ 1 pt Correcto!

✓ + 0 pts **Respuesta: d)** *Utilizar Domain Adaptation mediante técnicas como Adversarial Training para alinear las distribuciones de los dominios de entrenamiento (light-field) y de prueba (SEM).*

Domain Adaptation con Adversarial Training permite al modelo aprender representaciones que son invariantes a las diferencias entre los dominios de entrenamiento y prueba, facilitando la extrapolación a imágenes de SEM sin requerir una gran cantidad de datos específicos de este nuevo dominio.

Question 8

+ 2 pts Channel Attention Correcto!

+ 2 pts Spatial Attention Correcto!

+ 1 pt Integración Correcto!

+ 0 pts

```
import torch
import torch.nn as nn

class ChannelAttention(nn.Module):
    def __init__(self, in_chan):
        super(ChannelAttention, self).__init__()
        self.MLP = nn.Conv2d(in_chan, in_chan, kernel_size=1)
        self.ReLU = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avgPool = torch.mean(x, dim=(2, 3), keepdim=True)
        maxPool, _ = torch.max(x, dim=(2, 3), keepdim=True)
        out = self.MLP(avgPool) + self.MLP(maxPool)
        return self.sigmoid(out) * x

class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()
        self.conv = nn.Conv2d(2, 1, kernel_size=kernel_size, padding=kernel_size // 2)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avgPool = torch.mean(x, dim=1, keepdim=True)
        maxPool, _ = torch.max(x, dim=1, keepdim=True)
        concat = torch.cat([avgPool, maxPool], dim=1)
        out = self.conv(concat)
        return self.sigmoid(out) * x

class CBAM(nn.Module):
    def __init__(self, in_chan, kernel_size=7):
        super(CBAM, self).__init__()
        self.chAttn = ChannelAttention(in_chan)
        self.spAttn = SpatialAttention(kernel_size)

    def forward(self, x):
        x = self.chAttn(x)
        x = self.spAttn(x)
        return x
```

✓ + 1.5 pts Click here to replace this description.

✓ + 1.5 pts Click here to replace this description.

+ 0.5 pts Click here to replace this description.

+ 1 pt Click here to replace this description.

+ 1 pt Click here to replace this description.

Question 9

✓ + 1 pt Modo entrenamiento Correcto!

+ 1 pt Conv 3×3 en Modo evaluación Correcto!

+ 1 pt Conv 1×1 en Modo evaluación Correcto!

+ 1 pt Conexion residual en Modo evaluación Correcto!

+ 1 pt Batch normalization en Modo evaluación Correcto!

+ 0 pts

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class RepVGGBlock(nn.Module):
    def __init__(self, channels):
        super(RepVGGBlock, self).__init__()
        self.conv3 = nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn3 = nn.BatchNorm2d(channels)
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=1, stride=1, padding=0, bias=False)
        self.bn1 = nn.BatchNorm2d(channels)
        self.bn0 = nn.BatchNorm2d(channels)
        self.ReLU = nn.ReLU(inplace=True)

    def forward(self, x):
        out_3x3 = self.bn3(self.conv3(x))
        out_1x1 = self.bn1(self.conv1(x))
        out_id = self.bn0(x)
        out = self.ReLU(out_3x3 + out_1x1 + out_id, inplace=True)
        return out

class RepVGGBlockDeploy(nn.Module):
    def __init__(self, channels):
        super(RepVGGBlockDeploy, self).__init__()
        self.fused_conv = nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=True)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.relu(self.fused_conv(x))

# Fusiona una conv con su BN
def fuse_conv_bn(conv, bn):
    W = conv.weight # [channels, channels, k, k]
    gamma = bn.weight
    beta = bn.bias
    mean = bn.running_mean
    var = bn.running_var
    eps = bn.eps
    std = torch.sqrt(var + eps)

    W_fused = W * (gamma / std).reshape(-1, 1, 1, 1)
    b_fused = beta - mean * (gamma / std)
    return W_fused, b_fused

# Expande un kernel 1x1 a 3x3
def expand_1x1_to_3x3(W1):
```

```

outC, inC, _, _ = W1.shape
W1_expanded = torch.zeros((outC, inC, 3, 3), dtype=W1.dtype, device=W1.device)
W1_expanded[:, :, 1, 1] = W1[:, :, 0, 0]
return W1_expanded

# Crea el kernel identidad y lo fusiona con su BN
def fuse_identity_bn(channels, bn):
    # Crear el kernel identidad
    I = torch.eye(channels).view(channels, channels, 1, 1).to(bn.weight.device)

    # Extraer parámetros de BN:
    gamma = bn.weight
    beta = bn.bias
    mean = bn.running_mean
    var = bn.running_var
    eps = bn.eps

    # Calcular la desviación estándar:
    std = sqrt(var + eps)

    # Fusionar
    W_fused = I * (gamma / std).reshape([channels, 1, 1, 1])
    b_fused = beta - mean * (gamma / std)

    return W_fused, b_fused

# Función para convertir un bloque de entrenamiento a versión evaluación
def repVGG_convert_block(block):
    channels = block.conv3.in_channels

    # Fusionar rama 3x3
    W3, b3 = fuse_conv_bn(block.conv3, block.bn3)

    # Fusionar rama 1x1 y expandir a 3x3
    W1, b1 = fuse_conv_bn(block.conv1, block.bn1)
    W1_3x3 = expand_1x1_to_3x3(W1)

    # Fusionar rama identidad y expandir a 3x3
    W_id, b_id = fuse_identity_bn(channels, block.bn0)
    W_id_3x3 = expand_1x1_to_3x3(W_id)

    # Sumar los tres kernels y biases
    W_fused = W3 + W1_3x3 + W_id_3x3
    b_fused = b3 + b1 + b_id

    # Emplear los pesos fusionados
    deploy_block = RepVGGBlockDeploy(channels)
    deploy_block.fused_conv.weight.data.copy_(W_fused)
    deploy_block.fused_conv.bias.data.copy_(b_fused)
    return deploy_block

```

+ 1 pt [Click here to replace this description.](#)

💬 Realmente no entiendo lo que hiciste. Si me explicas, tal vez podría actualizar el puntaje.

Question 10

ROC vs AP

1.5 / 5 pts

+ 5 pts Correcto!

+ 0 pts Sea un conjunto de datos con N muestras totales; con $P = pn$ muestras positivas y $N = (1 - p)n$ muestras negativas, donde p es la proporción de positivos.

Tenemos:

$$TPR (\text{Recall}) = \frac{TP}{TP+FN}$$

$$FPR = \frac{FP}{FP+TN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

Observamos que:

$$TP = TPR \cdot P = TPR \cdot pn$$

$$FP = FPR \cdot N = FPR \cdot (1 - p)n$$

Entonces, la precisión se puede expresar como:

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{TPR \cdot pn}{TPR \cdot pn + FPR \cdot (1-p)n} = \frac{TPR \cdot p}{TPR \cdot p + FPR \cdot (1-p)} = \frac{TPR}{TPR + \left(\frac{1-p}{p}\right)FPR}$$

La expresión $\frac{1-p}{p}$ describe el grado de desbalanceo entre la clase negativa y la positiva.

Si el conjunto de datos está balanceado, se tiene $p=0.5$ y por tanto $\frac{1-p}{p} = \frac{0.5}{0.5} = 1$. Por lo tanto:

$$\text{Precision} = \frac{TPR}{TPR+FPR}$$

lo que indica que la relación entre TPR y FPR se refleja directamente en la precisión. En este escenario, la curva ROC y la curva AP serán comparables.

Si el conjunto está desbalanceado, por ejemplo $p = 0.01$, entonces $\frac{1-p}{p} = \frac{0.99}{0.01} = 99$. Por lo tanto:

$$\text{Precision} = \frac{TPR}{TPR+99 \cdot FPR}$$

Esto implica que aun valores pequeños de FPR se amplifican por el factor 99, reduciendo drásticamente la precisión. En otras palabras, el impacto de los falsos positivos es muy significativo cuando la clase positiva es muy rara.

En el caso de la curva ROC, dado que el FPR se calcula sobre el conjunto de negativos (que es muy grande en datos desbalanceados), incluso un número alto de falsos positivos puede resultar en un FPR bajo. Esto puede llevar a una interpretación optimista del rendimiento del clasificador; ya que la curva ROC no refleja adecuadamente la dificultad de distinguir la clase minoritaria.

Por otro lado, para el caso de la curva AP, la fórmula

$$\text{Precision} = \frac{TPR}{TPR + \left(\frac{1-p}{p}\right)FPR}$$

muestra que la precisión depende fuertemente del desbalance ($\frac{1-p}{p}$). Esto significa que la curva AP captura de manera más realista el rendimiento del clasificador en la detección de la clase positiva, penalizando fuertemente cualquier incremento en falsos positivos en escenarios desbalanceados.

---PR

+ 2 pts Click here to replace this description.

+ 1 pt Click here to replace this description.

✓ **+ 1.5 pts** Click here to replace this description.

+ 3 pts Click here to replace this description.

Question 11

Puntitos

1 / 4 pts

+ 2 pts +2pts

✓ + 1 pt +1pts

+ 0 pts tu no :':c

+ 4 pts Click here to replace this description.

+ 3 pts Click here to replace this description.

Apellidos: Méndez Lázaro

Nombres: Luis Fernando

Fecha: 28/01/2025

Nota:

Indicaciones:

La Duración es de 40 minutos.

La evaluación consta de 10 preguntas.

Las preguntas de opción múltiple valen 1.0 punto.

- Se entrena una red con learning rate schedule que incrementa linealmente el LR en las primeras 10 épocas (warm-up), seguido de un decaimiento tipo coseno. ¿Cuál es la principal ventaja de este enfoque respecto a un step decay?
 - Evita caer en mínimos locales mediante oscilaciones controladas. ☒
 - ☒ Acelera la convergencia inicial y ajusta finamente el LR cerca de óptimos. ☒
 - Minimiza la sensibilidad a la inicialización de parámetros. ☒
 - Reduce el riesgo de overfitting al final del entrenamiento. ☒
 - Permite un mayor LR máximo que el step decay. ☒
- ¿Cuál es la ventaja ROI Align frente a ROI Pooling? mark RCNN
 - ☒ Elimina la cuantización de coordenadas mediante interpolación bilineal. ☒
 - Aumenta la resolución espacial de los features mediante upsampling. ☒
 - Utiliza una region proposal network más precisa. ☒
 - Combina información de múltiples niveles de feature pyramids. ☒
 - Aplica label smoothing a las máscaras de segmentación. ☒
- ¿Cuál de las siguientes alternativas define mejor a una representación equivariante?
 - La salida de la red no cambia ante la transformación aplicada a la entrada. ☒
 - ☒ La representación se ajusta proporcionalmente a la magnitud de la transformación aplicada, asegurando una reducción de dimensionalidad óptima. ☒
 - La representación refleja la misma transformación aplicada a la entrada, permitiendo mantener información sobre la variación estructural. ☒
 - Garantiza un incremento monótonico en la distancia euclidiana del embedding cuando la transformación es mayor. ☒
 - Una red equivariante es un Lipschitz embedding. ☒
- Una característica de emplear transposed convolution es:
 - Suavizar los bordes de los objetos. ☒
 - Generar features invariantes a traslaciones. ☒
 - Limitar la capacidad de reconstrucción a texturas simples. ☒
 - Introduce artefactos (checkerboard artifacts). ☒
 - ☒ Preserva información de detalle procesada en el Encoder. ☒
- ¿Cuál es la función de los object queries en DETR (Detection Transformer)?
 - Generar los mapas de características piramidales necesarios para la detección multi-dimensional. ☒
 - Definir los anclajes (anchors) dinámicos basados en la distribución de píxeles de entrada. ☒
 - Reemplazar por completo el concepto de Non-Maximum Suppression. ☒
 - Transformar las coordenadas de las bounding boxes a un espacio latente, reduciendo la dimensionalidad para el post-procesamiento. ☒
 - ☒ Servir como un conjunto fijo de embeddings que es procesado con la salida del encoder para predecir las bounding boxes y las clases. ☒
- ¿Cuál de estas arquitecturas garantiza equivarianza a traslaciones?
 - MLP con activaciones ReLU. ☒
 - ☒ Self-attention con positional encoding. ☒
 - CNN con max-pooling. ☒
 - Self-attention sin positional encoding. ☒
 - CNN sin max-pooling. ☒
- Un equipo de investigación entrena un modelo de Deep Learning para clasificar microorganismos (bacterias y hongos) a partir de imágenes obtenidas mediante microscopía de campo claro (light-field). El dataset cubre varias especies, distintas condiciones de cultivo y algunas variaciones de contraste y brillo para aumentar la robustez. Sin embargo, el modelo entrenado es utilizado en imágenes obtenidas por un microscopio de barrido de electrones (SEM), con características visuales muy diferentes. ¿Cuál de los siguientes enfoques sería el más adecuado para enfrentar este problema, es decir, que el modelo logre

class

out of distri

extrapolar efectivamente a imágenes muy distintas a las que fueron usadas en el entrenamiento?

a) Aplicar Data Augmentation extensivo, incluyendo transformaciones que simulen las características del microscopio SEM, como cambios en la textura y el contraste.

b) Implementar Transfer Learning utilizando un modelo pre-entrenado en un dataset de imágenes obtenidas por microscopía SEM antes de ajustarlo con el dataset actual. ~~X~~

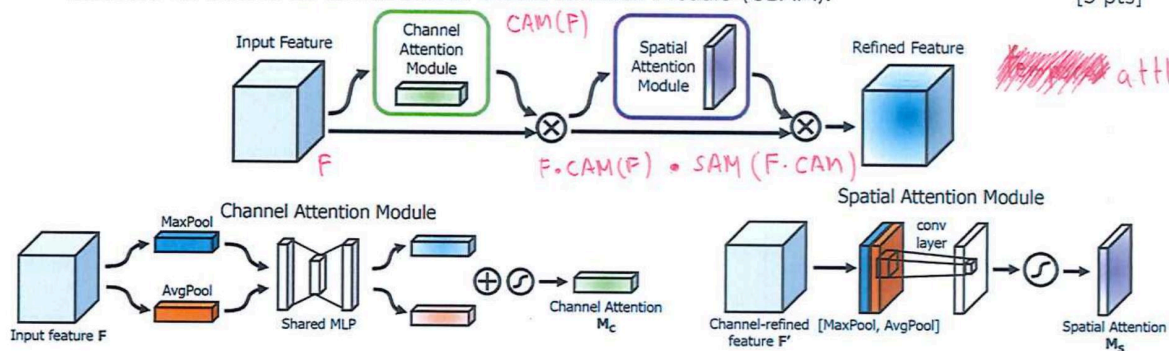
c) Emplear Few-Shot Learning para entrenar el modelo con un pequeño conjunto de imágenes SEM, permitiendo al modelo adaptarse rápidamente a las nuevas características visuales.

d) Utilizar Domain Adaptation mediante técnicas como Adversarial Training para alinear las distribuciones de los dominios de entrenamiento (light-field) y de prueba (SEM). ~~X~~

e) Congelar todas las capas del backbone y únicamente entrenar las capas superiores con el nuevo dominio SEM, minimizando el riesgo de sobreajuste. ~~X~~

8. Implemente un módulo de Convolutional Block Attention Module (CBAM).

[5 pts]



Channel Attention M:

$init(n_channels, H, W):$

$pool1 = \text{MaxPool2d}(H \times W)$

$pool2 = \text{AvgPool2d}(H \times W)$

$fc = \text{Linear}(in_channels, in_channels)$

forward(F):

$out1 = pool1(F)$

$out2 = pool2(F)$

flatten out1, out2

$out1 = fc(out1)$

$out2 = fc(out2)$

return $F \cdot \text{relu}(out1 + out2)$

Global poolings!

Spatial Attention M:

$init(n_channels, out_channels, H, W):$

$pool1 = \text{MaxPool2d}(\dots)$

$pool2 = \text{AvgPool2d}(\dots)$

$conv = \text{Conv2d}(in_channels, out_channels)$

forward(F):

$out1 = pool1(F')$

$out2 = pool2(F')$

$out3 = \text{cat}([out1, out2])$

$out4 = \text{Conv}(out3)$

return $F \cdot \text{relu}(out4)$

Concatenation!

CBAM:

$init(in_channels, out_channels, H, W):$

$CAM = \text{channelAttentionM}(\dots)$

$SAM = \text{SpatialAttentionM}(\dots)$

forward(F):

return $(F * CAM(F) * SAM(F * CAM(F)))$

9. Proponga un pseudocódigo que implemente RepVGG. [5 pts]

RepVGG Training:

init (in_chan, out_chan):

conv3x3 = Conv2d(in_chan, out_chan, stride=2)

conv1x1 = Conv2d(in_chan, out_chan, stride=2)

forward (x):

out1 = F.relu(conv3x3(x) + conv1x1(x))

out2 = F.relu(conv3x3(out1) + conv1x1(out1) + out1)

out3 = F.relu(conv3x3(out2) + conv1x1(out2) + out2)

return F.relu(conv3x3(out3) + conv1x1(out3) + out3)

RepVGG Inference:

init (in_chan, out_chan):

conv3x3 = # Combinar pesos

aprendidos de conv3x3 y conv1x1
de RepVGG Training :)

forward (x):

out1 = conv3x3(x)

out1 = F.relu(out1)

out2 = F.relu(conv3x3(out1))

out2 = F.relu(conv3x3(out1))

out3 = F.relu(conv3x3(out2))

return F.relu(conv3x3(out2))

10. Sobre las curvas ROC y AP, Demuestre matemáticamente cual es la más adecuada para enfrentar datos desbalanceados, y cual no. [5 pts]

AP (precision-recall curve) para datos desbalanceados

ROC para datos balanceados (métrica Global de rendimiento)

Precision = $TP / (TP + FP)$

recall = $TP / (TP + FN)$

